

We can use `filter` to convert successes into failures if the successful values don't match the given predicate. A common pattern is to transform an `Option` via calls to `map`, `flatMap`, and/or `filter`, and then use `getOrElse` to do error handling at the end:

```
val dept: String =
  lookupByName("Joe").
  map(_.dept).
  filter(_ != "Accounting").
  getOrElse("Default Dept")
```

`getOrElse` is used here to convert from an `Option[String]` to a `String`, by providing a default department in case the key "Joe" didn't exist in the `Map` or if Joe's department was "Accounting".

`orElse` is similar to `getOrElse`, except that we return another `Option` if the first is undefined. This is often useful when we need to chain together possibly failing computations, trying the second if the first hasn't succeeded.

A common idiom is to do `o.getOrElse(throw new Exception("FAIL"))` to convert the `None` case of an `Option` back to an exception. The general rule of thumb is that we use exceptions only if no reasonable program would ever catch the exception; if for some callers the exception might be a recoverable error, we use `Option` (or `Either`, discussed later) to give them flexibility.

As you can see, returning errors as ordinary values can be convenient and the use of higher-order functions lets us achieve the same sort of consolidation of error-handling logic we would get from using exceptions. Note that we don't have to check for `None` at each stage of the computation—we can apply several transformations and then check for and handle `None` when we're ready. But we also get additional safety, since `Option[A]` is a different type than `A`, and the compiler won't let us forget to explicitly defer or handle the possibility of `None`.

4.3.2 Option composition, lifting, and wrapping exception-oriented APIs

It may be easy to jump to the conclusion that once we start using `Option`, it infects our entire code base. One can imagine how any callers of methods that take or return `Option` will have to be modified to handle either `Some` or `None`. But this doesn't happen, and the reason is that we can *lift* ordinary functions to become functions that operate on `Option`.

For example, the `map` function lets us operate on values of type `Option[A]` using a function of type `A => B`, returning `Option[B]`. Another way of looking at this is that `map` turns a function `f` of type `A => B` into a function of type `Option[A] => Option[B]`. Let's make this explicit:

```
def lift[A,B](f: A => B): Option[A] => Option[B] = _ map f
```

This tells us that any function that we already have lying around can be transformed (via `lift`) to operate *within the context of* a single `Option` value. Let's look at an example:

```
val absO: Option[Double] => Option[Double] = lift(math.abs)
```

The `math` object contains various standalone mathematical functions including `abs`, `sqrt`, `exp`, and so on. We didn't need to rewrite the `math.abs` function to work with optional values; we just lifted it into the `Option` context after the fact. We can do this for *any* function. Let's look at another example. Suppose we're implementing the logic for a car insurance company's website, which contains a page where users can submit a form to request an instant online quote. We'd like to parse the information from this form and ultimately call our `rate` function:

```
/**
 * Top secret formula for computing an annual car
 * insurance premium from two key factors.
 */
def insuranceRateQuote(age: Int, numberOfSpeedingTickets: Int): Double
```

We want to be able to call this function, but if the user is submitting their age and number of speeding tickets in a web form, these fields will arrive as simple strings that we have to (try to) parse into integers. This parsing may fail; given a string, `s`, we can attempt to parse it into an `Int` using `s.toInt`, which throws a `NumberFormatException` if the string isn't a valid integer:

```
scala> "112".toInt
res0: Int = 112

scala> "hello".toInt
java.lang.NumberFormatException: For input string: "hello"
    at java.lang.NumberFormatException.forInputString(...)
    ...
```

Lifting functions

```
lift(math.abs):    Option[Double] => Option[Double]
                  ↖               ↗
math.abs:         Double => Double

lift(f) returns a function which maps None to None
and applies f to the contents of Some. f need
not be aware of the Option type at all.
```

Let's convert the exception-based API of `toInt` to `Option` and see if we can implement a function `parseInsuranceRateQuote`, which takes the age and number of speeding tickets as strings, and attempts calling the `insuranceRateQuote` function if parsing both values is successful.

Listing 4.3 Using Option

```
def parseInsuranceRateQuote(
  age: String,
  numberOfSpeedingTickets: String): Option[Double] = {
  val optAge: Option[Int] = Try(age.toInt)
  val optTickets: Option[Int] = Try(numberOfSpeedingTickets.toInt)
```

The `toInt` method is available on any `String`.

Note: This discards information about the error. We'll improve on this later in the chapter.

```
insuranceRateQuote(optAge, optTickets)
}
```

Doesn't type check!
See following discussion.

```
def Try[A](a: => A): Option[A] =
  try Some(a)
  catch { case e: Exception => None }
```

We accept the A argument non-strictly, so we can catch any exceptions that occur while evaluating a and convert them to None.

The Try function is a general-purpose function we can use to convert from an exception-based API to an Option-oriented API. This uses a non-strict or lazy argument, as indicated by the `=> A` as the type of `a`. We'll discuss laziness much more in the next chapter.

But there's a problem—after we parse `optAge` and `optTickets` into `Option[Int]`, how do we call `insuranceRateQuote`, which currently takes two `Int` values? Do we have to rewrite `insuranceRateQuote` to take `Option[Int]` values instead? No, and changing `insuranceRateQuote` would be entangling concerns, forcing it to be aware that a prior computation may have failed, not to mention that we may not have the ability to modify `insuranceRateQuote`—perhaps it's defined in a separate module that we don't have access to. Instead, we'd like to lift `insuranceRateQuote` to operate in the context of two optional values. We could do this using explicit pattern matching in the body of `parseInsuranceRateQuote`, but that's going to be tedious.



EXERCISE 4.3

Write a generic function `map2` that combines two `Option` values using a binary function. If either `Option` value is `None`, then the return value is too. Here is its signature:

```
def map2[A,B,C](a: Option[A], b: Option[B])(f: (A, B) => C): Option[C]
```

With `map2`, we can now implement `parseInsuranceRateQuote`:

```
def parseInsuranceRateQuote(
  age: String,
  numberOfSpeedingTickets: String): Option[Double] = {
  val optAge: Option[Int] = Try { age.toInt }
  val optTickets: Option[Int] = Try { numberOfSpeedingTickets.toInt }
  map2(optAge, optTickets)(insuranceRateQuote)
}
```

Functions accepting a single argument may be called with braces instead of parentheses; this is equivalent to `Try(age.toInt)`.

If either parse fails, this will immediately return `None`.

The `map2` function means that we never need to modify any existing functions of two arguments to make them “Option-aware.” We can lift them to operate in the context of `Option` after the fact. Can you already see how you might define `map3`, `map4`, and `map5`? Let's look at a few other similar cases.

 EXERCISE 4.4

Write a function `sequence` that combines a list of `Options` into one `Option` containing a list of all the `Some` values in the original list. If the original list contains `None` even once, the result of the function should be `None`; otherwise the result should be `Some` with a list of all the values. Here is its signature:³

```
def sequence[A](a: List[Option[A]]): Option[List[A]]
```

Sometimes we'll want to map over a list using a function that might fail, returning `None` if applying it to any element of the list returns `None`. For example, what if we have a whole list of `String` values that we wish to parse to `Option[Int]`? In that case, we can simply sequence the results of the map:

```
def parseInts(a: List[String]): Option[List[Int]] =  
  sequence(a map (i => Try(i.toInt)))
```

Unfortunately, this is inefficient, since it traverses the list twice, first to convert each `String` to an `Option[Int]`, and a second pass to combine these `Option[Int]` values into an `Option[List[Int]]`. Wanting to sequence the results of a map this way is a common enough occurrence to warrant a new generic function, `traverse`, with the following signature:

```
def traverse[A, B](a: List[A])(f: A => Option[B]): Option[List[B]]
```

 EXERCISE 4.5

Implement this function. It's straightforward to do using `map` and `sequence`, but try for a more efficient implementation that only looks at the list once. In fact, implement `sequence` in terms of `traverse`.

For-comprehensions

Since lifting functions is so common in Scala, Scala provides a syntactic construct called the *for-comprehension* that it expands automatically to a series of `flatMap` and `map` calls. Let's look at how `map2` could be implemented with for-comprehensions.

³ This is a clear instance where it's not appropriate to define the function in the OO style. This shouldn't be a method on `List` (which shouldn't need to know anything about `Option`), and it can't be a method on `Option`, so it goes in the `Option` companion object.